

Assignment 2 Report: Analyzing and Implementing Divide-and-Conquer

Algorithms

Anushka Nanaware

STUDENT ID:

Department of Computer Science

University of the Cumberlands

MSCS532 - M80 Algorithms and Data Structures

Dr. Micahel Solomon

January 31, 2026

Divide-and-conquer is a way of solving problems by breaking a big problem into smaller parts, solving each part one by one, and then putting the results together to get the final answer.

This approach is widely used because it simplifies complex problems and often improves

efficiency compared to basic iterative methods (Cormen et al., 2022). In this assignment, Merge and Quick Sort, which are two divide-and-conquer sorting algorithms, are analyzed and then implemented. Merge Sort consistently divides the input into equal halves and provides predictable performance, while Quick Sort partitions the input around a pivot and often performs faster in practice but can degrade in the worst case (Cormen et al., 2022). The assignment examines these algorithms through theoretical analysis using asymptotic notation and recurrence relations, as well as practical evaluation using Python implementations tested on sorted, reverse-sorted, and random datasets, to better understand their real-world behavior.

Merge Sort is used to sort a list of elements in ascending order. The problem it solves is arranging an unordered collection of values into a specific order so that each element is less than or equal to the next. Sorting is a fundamental operation in computer science and is commonly required in searching, data processing, and optimization tasks (Cormen et al., 2022). The key idea behind Merge Sort is to repeatedly divide the input list into smaller parts until each sublist contains only one element, and a list which has a single element is already sorted. Once the list has been divided into these smallest units, the algorithm begins merging the sublists back together in sorted order. During the merge step, two sorted sublists are combined by repeatedly selecting the smaller element from the front of each list until all elements have been merged.

The main steps are to get two equal halves by dividing into the input list, then use Merge Sort and then sort each of the half and then merge them into one sorted list and then this continues until everything in the list is sorted. Because Merge Sort always divides the list evenly and performs the same operations regardless of the input order, its behavior remains consistent across different datasets (Cormen et al., 2022).

Time Complexity Analysis

The **time complexity of Merge Sort** is consistent across all types of input because the algorithm always divides the list into two equal halves and performs the same number of operations during the merge step. As a result, its performance does not depend on whether the input data is already sorted, reverse sorted, or randomly ordered (Cormen et al., 2022). In the **best case**, Merge Sort still divides the input into smaller sublists and merges them back together. Even if the input list is already sorted, the algorithm performs all recursive calls and merge operations and thus $\Omega(n \log n)$ is the best-case time complexity of Merge Sort. In the **worst case**, Merge Sort follows the same sequence of operations as in the best case. The list is repeatedly divided into halves, and merging all elements requires linear time at each level of recursion. Thus, the worst-case time complexity of Merge Sort is $O(n \log n)$ and the algorithm follows the same steps in both the best and worst cases, its average-case time complexity is also $O(n \log n)$. This predictable performance makes Merge Sort a reliable sorting algorithm, especially when consistent execution time is required (Cormen et al., 2022).

Recurrence Relation and Solution

The running time of Merge Sort can be described using a recurrence relation because the algorithm solves smaller subproblems recursively. At each step, the input list of size n is divided into two sub lists of size $n/2$. Each sub list is sorted recursively, and then the two sorted sublists are merged. The merge operation processes all n elements, which takes linear time (Cormen et al., 2022) and thus for merge sort the recurrence relation is: $T(n) = 2T(n/2) + O(n)$. Here, the

term $2T(n/2)$ represents the time required to recursively sort the two halves of the list, and $O(n)$ represents the time required to merge the two sorted halves.

Solution Using the Substitution Method

To solve the recurrence relation using the substitution method, we assume a solution and then verify that it satisfies the recurrence. If we consider the running time of Merge Sort is $O(n \log n)$ then at each level of recursion, the problem is divided into two smaller problems, and the total amount of work done at each level is proportional to n . Since the number of recursive levels is proportional to $\log n$, the total running time becomes $O(n \log n)$. This confirms that the assumed solution satisfies the recurrence of relation.

Solution Using the Recursion Tree Method

Using the recursion tree method, the recurrence can be visualized as a tree where each node represents a recursive call. At the top level, the cost of merging is $O(n)$. At the next level, there are two subproblems of size $n/2$, and the total cost at that level is also $O(n)$. This pattern continues for each level of the tree as $\log n$ is the height of the recursion tree and each level contributes $O(n)$ work and thus gives the total running time $O(n \log n)$.

Solution Using the Master Method

There is direct method that help solve the recurrence relations and that is the Master Method, and they are of the the form $T(n) = aT(n/b) + f(n)$ and the values for merge sort are $a = 2$, $b = 2$, and $f(n) = O(n)$. Since $f(n)$ matches $O(n^{\log_b a})$, which is $O(n)$, this recurrence

falls under Case 2 of the Master Method and thus the solution for the recurrence relation becomes **$O(n \log n)$** (Cormen et al., 2022).

Practical Implications

Merge Sort is widely used in practice because of its predictable and stable performance. Since its time complexity is $O(n \log n)$ in the best, average, and worst cases, it provides consistent execution time regardless of the input order. This makes Merge Sort a good choice in situations where performance reliability is more important than raw speed (Cormen et al., 2022). Merge Sort is a **stable** sorting algorithm which is another advantage, which means that elements with the same value keep their original order after sorting. This property is useful in applications like database systems and multi-level sorting, where maintaining the original order is important. However, Merge Sort also requires extra memory to store temporary sublists during the merging process. This extra space usage can be a disadvantage when working with very large datasets or memory-constrained systems. Despite this limitation, Merge Sort is commonly used in external sorting and large-scale data processing because it handles large inputs efficiently and consistently (Cormen et al., 2022).

QUICK SORT

Quick Sort is also a divide-and-conquer sorting algorithm which is again used to arrange a list of elements in ascending order. Like Merge Sort, it solves the problem of ordering data so that elements can be processed, searched, or analyzed more efficiently. Quick Sort is well known for its strong performance in practice and is commonly used in real-world applications (Cormen et al., 2022). The main idea behind Quick Sort is to select a **pivot element** from the list and

partition the remaining elements into two groups: those smaller than or equal to the pivot and those greater than the pivot. After the list is divided, the pivot element is placed in the position where it belongs in the sorted list. The algorithm then repeats the same steps on the elements to the left and right of the pivot until the entire list is sorted.

The main steps of the Quick Sort algorithm are to choose one element as the pivot., rearrange the list so that elements smaller than or equal to the pivot come before it, and elements larger than the pivot come after it, apply Quick Sort again to the left and right parts of the list and stop when the sublists have zero or one element. Unlike Merge Sort, Quick Sort does not always split the list into equal halves, which is why its performance can change depending on the input data. Its performance depends on how well the pivot divides the list. When the pivot splits the list evenly, the algorithm performs efficiently. However, if the pivot selection is poor, the algorithm can become unbalanced, leading to slower performance (Cormen et al., 2022).

Time Complexity Analysis

The time complexity of Quick Sort depends heavily on how the pivot element divides the input list during each recursive step. Unlike Merge Sort, Quick Sort does not always split the list evenly, which causes its performance to vary across best, average, and worst cases (Cormen et al., 2022). In the best case, the pivot splits the list into two almost equal parts at each step. This keeps the number of recursive calls small, and each step processes all n elements during partitioning and thus the best-case time complexity is $\Omega(n \log n)$. And in the average case when the pivot is chosen carefully or randomly, the list is usually divided fairly evenly. This allows Quick Sort to run efficiently, with an average-case time complexity of $O(n \log n)$, which is why it is often faster than many other sorting algorithms in real use (Cormen et al., 2022). In the

worst case, however, the pivot creates very uneven partitions, such as when the smallest or largest element is chosen for an already sorted or reverse-sorted list, leading to much slower performance. In this case, one sublist contains $n - 1$ elements while the other contains zero elements. This leads to deep recursion and significantly more comparisons, resulting in a worst-case time complexity of $O(n^2)$. This worst-case behavior was observed during implementation when Quick Sort was applied to ordered datasets without randomized pivot selection.

Recurrence Relation and Solution

The running time of Quick Sort can also be expressed using recurrence relations because the algorithm works recursively on smaller sublists. At each step, the list is partitioned around a pivot element, and the algorithm is recursively applied to the sublists on either side of the pivot. The time required to partition the list is proportional to the size of the list, or $O(n)$ (Cormen et al., 2022).

Best and Average Case Recurrence

In both the best and average cases, the list is divided by pivot into two approximately equal halves and we get the recurrence relation: $T(n) = 2T(n/2) + O(n)$. Here, $2T(n/2)$ represents the recursive sorting of the two halves, and $O(n)$ represents the cost of partitioning the list.

Substitution Method

Assuming a solution of $O(n \log n)$, the recurrence satisfies this assumption because each level of recursion performs linear work, and the number of levels grows logarithmically and thus gives $O(n \log n)$.

Recursion Tree Method

During partitioning, each level of the recursion performs $O(n)$ work and $\log n$ is the height of the tree and $\Theta(n \log n)$ is the total work that is done across all the levels.

Master Method

For this recurrence, $a = 2$, $b = 2$, and $f(n) = O(n)$. This matches Case 2 of the Master Method, resulting in a solution of $\Theta(n \log n)$ (Cormen et al., 2022).

Worst Case Recurrence

In the worst case, the pivot produces a highly unbalanced partition, where one sublist contains $n - 1$ elements and the other contains zero elements. This results in the recurrence: $T(n) = T(n - 1) + O(n)$ and this doesn't satisfy the Master Method. Solving it using repeated substitution shows that the total running time grows quadratically, leading to a solution of $O(n^2)$. This explains why Quick Sort can perform poorly on already sorted or reverse-sorted inputs when pivot selection is not handled carefully.

Practical Implications

Quick Sort is widely used in practice because it is fast and efficient on average and has low memory overhead. Since it performs sorting in place, it requires very little additional memory compared to algorithms like Merge Sort. This makes Quick Sort a popular choice in systems where memory usage is a concern (Cormen et al., 2022). In real-world applications, Quick Sort often outperforms other comparison-based sorting algorithms due to its good cache performance and simple operations. When pivots are chosen randomly or carefully, the algorithm

usually produces balanced partitions, resulting in an average-case time complexity of $O(n \log n)$. This behavior was observed during implementation when randomized pivot selection was used. However, Quick Sort has an important limitation: its performance can degrade significantly in the worst case. If poor pivot choices are made, such as consistently selecting the smallest or largest element, the algorithm can experience deep recursion and quadratic running time. This worst-case behavior was encountered during testing on sorted and reverse-sorted datasets before pivot randomization was applied. As a result, practical implementations of Quick Sort often include strategies such as randomized pivot selection to avoid this issue (Cormen et al., 2022).

Code Implementation Summary

Both Merge Sort and Quick Sort were implemented in Python using a recursive divide-and-conquer approach. The implementations closely follow the theoretical definitions of the algorithms, with each function designed to clearly reflect the divide, conquer, and combine steps discussed in the analysis sections. Merge Sort was implemented as a function that returns a new sorted list. The algorithm keeps dividing the input list into two smaller halves until each part has only one element and then these small lists are then combined back together in sorted order. This implementation prioritizes clarity and correctness and directly demonstrates the consistent recursive structure of Merge Sort.

Quick Sort was implemented using an in-place recursive approach with a wrapper function to return a new sorted list for fair comparison. During initial testing, a fixed pivot selection strategy resulted in worst-case behavior on sorted and reverse-sorted inputs, leading to deep recursion. To address this, randomized pivot selection was introduced. This modification significantly improved practical performance and prevented worst-case recursion during

benchmarking. A separate benchmarking script was created to measure execution time for both algorithms across different input sizes and data distributions. The benchmarking process ensured that both algorithms were tested under identical conditions to allow a fair and meaningful comparison.

Datasets Used

To evaluate the practical performance of Merge Sort and Quick Sort, three different types of input datasets were used: sorted, random, and reverse-sorted data. These datasets were chosen to represent common and edge-case scenarios encountered in real-world applications. The sorted dataset consists of elements arranged in an ascending order. This dataset is particularly important for evaluating Quick Sort, as it can trigger worst-case behavior when poor pivot selection strategies are used. The reverse-sorted dataset contains elements arranged in descending order and presents similar challenges for pivot-based sorting algorithms. The random dataset contains elements generated in random order and represents typical, real-world input. Both algorithms generally perform well on this type of data. Each dataset was tested across multiple input sizes to observe how execution time scales as the size of the input increases.

PERFORMANCE METRICS TABLE

	algorithm	data_type	size	time_seconds
1				
2	Merge Sort	sorted	500	0.0003156
3	Quick Sort	sorted	500	0.0002662
4	Merge Sort	reverse	500	0.0003109
5	Quick Sort	reverse	500	0.0002687
6	Merge Sort	random	500	0.0004733
7	Quick Sort	random	500	0.0003357
8	Merge Sort	sorted	1000	0.0006783
9	Quick Sort	sorted	1000	0.000577
10	Merge Sort	reverse	1000	0.0006513
11	Quick Sort	reverse	1000	0.0005896
12	Merge Sort	random	1000	0.0010675
13	Quick Sort	random	1000	0.000664
14	Merge Sort	sorted	2000	0.0014355
15	Quick Sort	sorted	2000	0.0013766
16	Merge Sort	reverse	2000	0.0014481
17	Quick Sort	reverse	2000	0.0013547
18	Merge Sort	random	2000	0.0022421
19	Quick Sort	random	2000	0.0015028
20	Merge Sort	sorted	5000	0.0038945
21	Quick Sort	sorted	5000	0.0033776
22	Merge Sort	reverse	5000	0.0038483
23	Quick Sort	reverse	5000	0.0035449
24	Merge Sort	random	5000	0.005575
25	Quick Sort	random	5000	0.0038439
26	Merge Sort	sorted	10000	0.0077351
27	Quick Sort	sorted	10000	0.0072391
28	Merge Sort	reverse	10000	0.0082255
29	Quick Sort	reverse	10000	0.0073888
30	Merge Sort	random	10000	0.0123694
31	Quick Sort	random	10000	0.0083108

The execution times for Merge Sort and Quick Sort were recorded using Python’s high-resolution timing functions. The results were collected across different input sizes and data distributions to evaluate how each algorithm scales in practice. The table summarizes representative measurements from the full dataset and highlights performance differences between the two algorithms under varying conditions

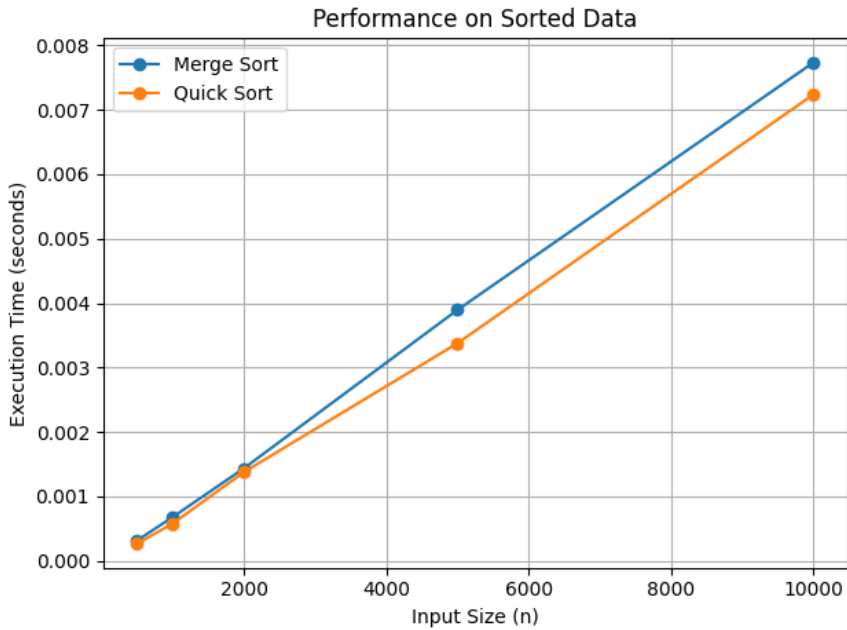


Figure 1. Execution time comparison of Merge Sort and Quick Sort on sorted input data across.



Figure 2. Execution time comparison of Merge Sort and Quick Sort on reverse-sorted input data across different input sizes.

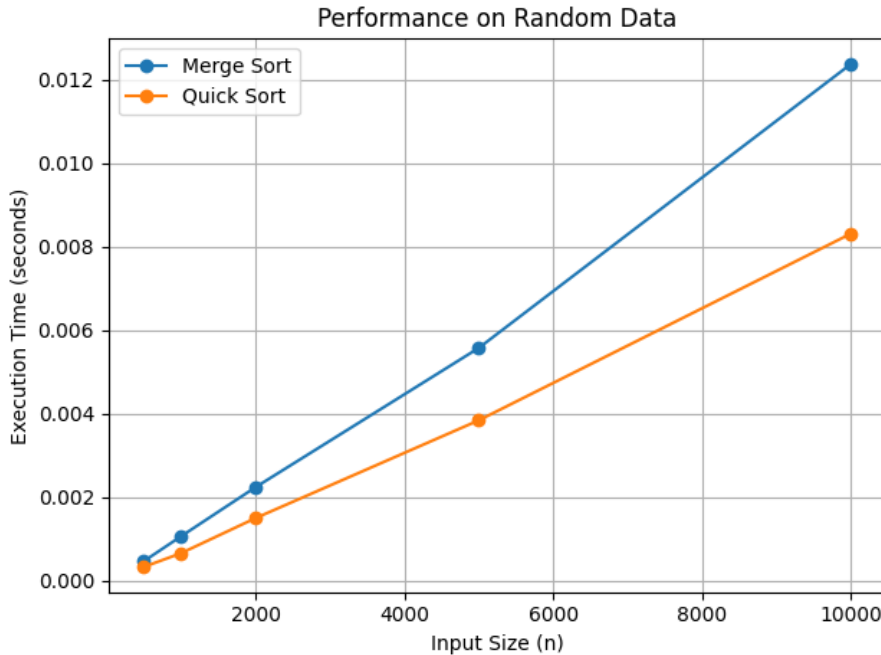


Figure 3. Execution time comparison of Merge Sort and Quick Sort on random input data across different input sizes.

Analysis of Results

The experimental results align closely with the theoretical analysis of both algorithms. Merge Sort demonstrates stable performance across all dataset types, with execution time increasing consistently as input size grows. Quick Sort shows strong performance on random data, often outperforming Merge Sort due to lower constant factors and efficient in-place operations. However, its performance is more sensitive to input order. When applied to sorted and reverse-sorted datasets without proper pivot handling, Quick Sort initially exhibited worst-case behavior. After introducing randomized pivot selection, the algorithm maintained efficient performance across all datasets. These results highlight the trade-off between theoretical

guarantees and practical efficiency. While Merge Sort offers predictable performance, Quick Sort can provide faster execution in practice when implemented with appropriate safeguards

This assignment analyzed Merge Sort and Quick Sort using both theoretical concepts and practical implementation. The study examined how these algorithms work, their time complexities using asymptotic notation and recurrence relations, and their performance on real datasets. Merge Sort showed consistent $O(n \log n)$ performance in all cases, making it a reliable choice despite its additional memory usage. Quick Sort performed faster on average, especially for random data, but its efficiency depended on pivot selection, with worst-case behavior avoided through randomized pivots. Overall, the experimental results aligned with theoretical expectations and emphasized the importance of combining algorithm analysis with practical implementation when selecting divide-and-conquer algorithms.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.